

Card Sathi

A credit-card recommendation engine that shows its working: rules decide, retrieval explains, a verifier checks.

Built as an agentic AI application over a structured catalogue, a policy corpus, and two language models. This report covers the market context, the architecture and why I chose it, the dataset I generated, how I evaluated it, and where it runs.

Author	Kamal Nayan
Date	02 October 2026
Code	https://github.com/Kamal-Nayan-Kumar/bank_sathi
Live demo	https://card-sathi.vercel.app
API	https://card-sathi-api.onrender.com

1 The market I built for

India had more than 100 million credit cards in force by 2024, and the number keeps climbing as banks push issuance beyond the metros. Yet only a single-digit share of adults holds one. The growth story and the penetration story point in opposite directions: cards are being sold aggressively into a population that mostly has no credit history, a thin file, or a score it does not understand.

Three facts about this market shaped every design decision in this project:

- **Eligibility is opaque.** Banks publish minimum income and score requirements per card, but a first-time applicant cannot tell which of fifty similar cards they qualify for — or why a rejection happened. The result is scattered applications, and every application leaves a hard enquiry on the file, which lowers the score it was meant to use.
- **The score system is unforgiving and slow-moving.** CIBIL scores run 300–900. A missed payment stays on file for years; a thin file means starting from secured and starter products. Advice like “take a loan to build your score” circulates widely and usually hurts the borrower. Any system giving guidance here has to be careful about what it suggests.
- **The catalogue is wide and samey.** Cashback, travel, fuel, dining, shopping, premium tiers — the differences that matter (caps, waiver thresholds, lounge conditions) hide in terms documents nobody reads. Comparing two cards honestly requires arithmetic on the customer's own spending, not a star rating.

That is the gap Card Sathi sits in: not another chatbot that chats about cards, but a decision engine that checks every card against the applicant's numbers, ranks what survives by rupee value, and shows the working. The audience I had in mind is the applicant with a thin file and real spending — exactly the

customer the current market serves worst.

2 Problem and what I built

The problem, stated plainly: **a customer cannot answer “which card can I actually get, and which of those is worth most to me?”** without either applying blindly or reading fifty terms documents. Existing comparison sites rank by sponsorship and ask nothing about spending; bank chatbots answer questions but make no decisions.

Card Sathi answers it in two parts:

- **Can you get it?** — a deterministic eligibility engine checks the company-wide rules once, then every card's published requirements against the profile. No model is involved. Failures carry reason codes and exact gaps (“Rs 6,000 more a month”, “25 score points short”).
- **Is it worth it to you?** — a scoring function reduces each eligible card to net annual rupee value on *your* spending, plus three smaller fit terms, and ranks them. The interface draws the four components as a proportional bar, so the order is visibly derived.

Around that core sit the language parts: an extractor that turns free text into a profile (asking one question at a time when something is missing), a retriever that grounds explanations in policy text, a generator that writes the summary, and a verifier that blocks anything the engine did not say. Two doors in — describe yourself in words, or fill the form — one hallway through.

3 Architecture and why I chose it

The single most important decision in this project is the trust hierarchy. Everything else follows from it:

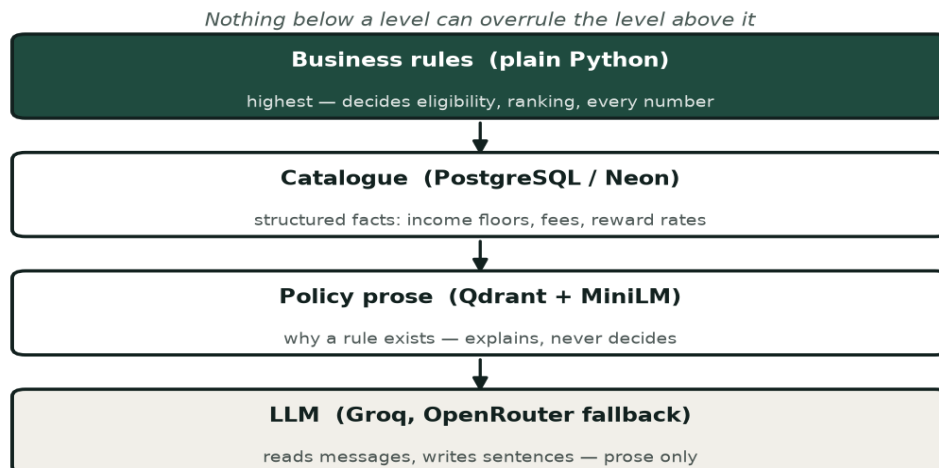


Figure 1 — nothing below a level can overrule the level above it. The model sits at the bottom on purpose.

Why it looks like this:

- **Rules in plain Python, not in a model.** “Minimum income $\leq 45,000$ ” is an exact numeric comparison. Embeddings are fuzzy by design and cannot reliably do it — and when they get it wrong there is nothing to inspect. The engine is ~600 lines I can test at every boundary.
- **PostgreSQL for the catalogue, Qdrant for the prose.** Card facts (income floors, fees, reward rates) live in SQL where they can be filtered exactly. Terms and policy guidance live in the vector store, where semantic search is genuinely useful. Mixing the two — embedding card rows and hoping the retriever does arithmetic — is the failure mode I designed this split to avoid.
- **The LLM handles language, never decisions.** It extracts profiles, asks follow-ups, and writes summaries from computed facts. It cannot mark a card eligible, change a rank, or write SQL — there is simply no tool or prompt path that lets it.
- **Thresholds live in one Python module, not a YAML file.** I started with policy.yaml as the source of truth and generated the RAG documents from it. That solved consistency but created a worse problem: a policy owner editing the markdown had to know about the YAML behind it. Now backend/app/thresholds.py holds every number, the markdown holds prose with no numbers at all, and a test fails the build if a number ever appears in the docs. Two artefacts that cannot disagree because they were never in the same place.
- **Embeddings run locally.** all-MiniLM-L6-v2 through fastembed — 22M parameters, fast on CPU. No embedding API key, no per-request cost, and retrieval quality never depends on a third-party key being present.

4 The LangGraph pipeline

I used LangGraph for the orchestration because the workflow has genuine branches: the profile may be incomplete, the gate may reject outright, there may be no eligible cards, and the final text must pass verification. Eight nodes, four conditional edges — a fixed graph, not an agent choosing tools. The “agentic” behaviour is that the *workflow* decides what happens next from the state.

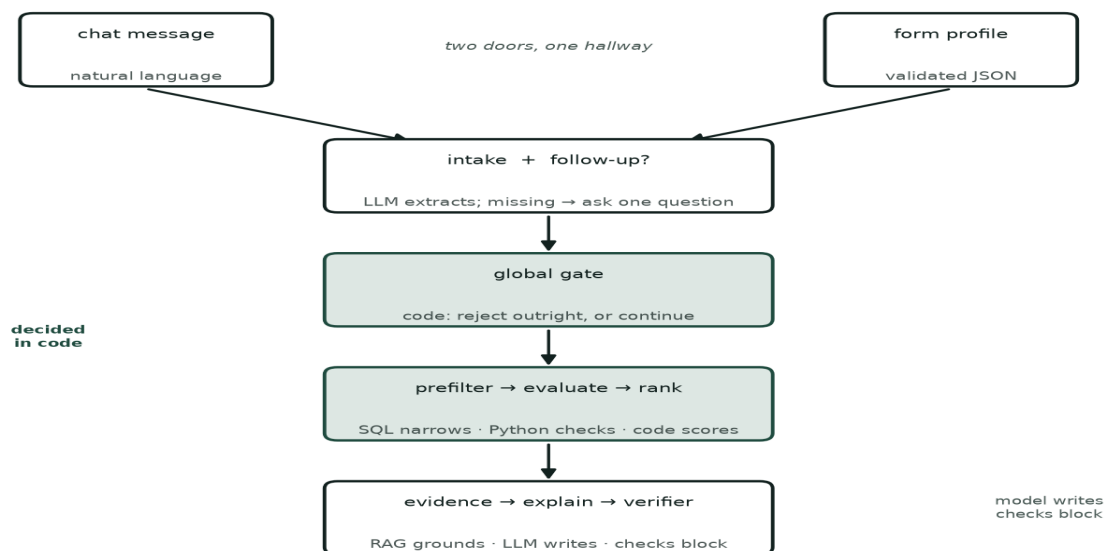


Figure 2 — the decision path. The model appears in exactly two nodes, neither of which can change a number.

What each node owns:

Node	Owns	Must never
intake	Message → partial profile. Masks PII first, leaves blanks null.	invent a value
followup	One question, then ends the turn.	ask for two things
build_profile	Partial → validated UserProfile. Fails loudly.	default an income to zero
gate	Company-wide screen: age, score floor, defaults, enquiries, usage.	consult a model
prefilter	Parameterised SQL narrows 120 cards to ~60.	take model-written SQL
evaluate	Every rule for every survivor, in Python. Returns reason codes + gaps.	stop at the first failure
rank	Net-value scoring, deterministic order.	break ties opaquely
evidence	RAG for the top cards and the rejection reasons only.	retrieve for cards nobody sees
explain	Prose from computed facts + chunks. Retries ≤ 2 , then template.	use a number not in the facts
verifier	12 checks: eligibility, order, numbers, PII, codes, evidence.	change a decision

State is a single TypedDict every node reads and partially updates, so a failure can be attributed to exactly one step — and the interface shows each step with its real timing on every result page.

5 LLM and RAG design

Two models, two jobs. Groq's openai/gpt-oss-120b is the primary for both extraction and explanation; when its free tier rate-limits — which it does, often — OpenRouter's free Nemotron endpoint takes over. I probed all 17 free OpenRouter endpoints before choosing it: only five honoured JSON mode, which extraction depends on. Failover on a 429 is immediate; retrying a saturated free tier just adds the backoff to the customer's wait, which I measured at ~40 wasted seconds before changing it.

Extraction is built to under-claim.

- The prompt says to leave fields null rather than guess. A wrong income produces a confident wrong recommendation; a blank produces one question.
- The regex fallback handles real formats (“1.2L”, “95k a month”, “8.5 LPA”) and runs the whole pipeline offline — which is also what CI tests.
- Model output is coerced into the Pydantic schema (“online_shopping” as a preference becomes “cashback”); what cannot be coerced is dropped, and only then does the regex path take over.

Retrieval supplies evidence, never answers.

retrieval never answers — it only supplies the sentences the answer cites

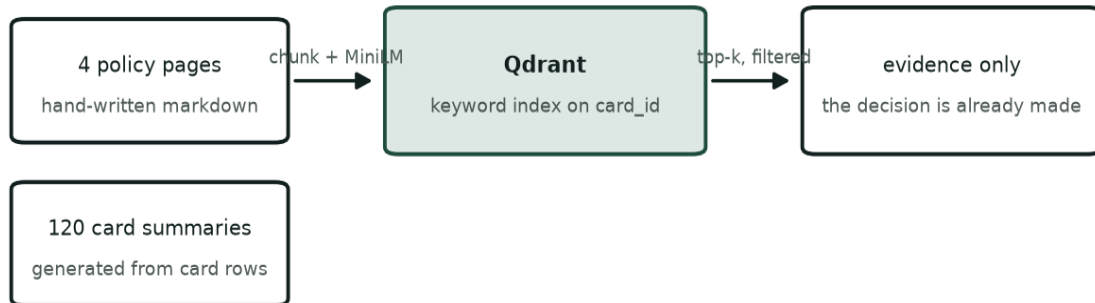


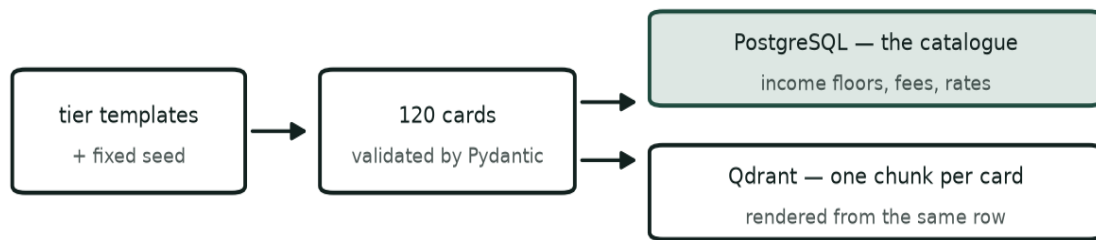
Figure 3 — the corpus and what retrieval is allowed to do with it.

- Four hand-written policy pages plus one chunk per card, rendered from the card's own database row — so a policy sentence cannot state an income minimum the engine does not enforce.
- Card queries are filtered by card_id (with a Qdrant keyword index — missing it 400s every filtered query, which I found out in production). The explanation for card A can never be grounded in card B's terms.
- The verifier's number check whitelists only engine-computed figures, retrieved policy text, and the customer's own numbers, with a 2% tolerance for honest rounding.

6 Dataset preparation

There was no catalogue to download, so I generated one — but not by asking a model for “120 credit cards”, which produces fees that contradict their own reward caps. Templates per segment (cashback, travel, fuel, dining, shopping, premium, beginner, business, lifestyle, student) crossed with four tiers, drawn from a fixed seed. Same seed, byte-identical catalogue, on any machine.

the card row is the single source of truth; both stores derive from it



300 profiles + 22 hand-written edge cases → ground truth computed by the engine itself

Figure 4 — one source of truth, two stores, one evaluation set.

- **120 cards, 10 segments × 4 tiers.** Every row validated by Pydantic. A few are withdrawn so the inactive-card path is real. Fields with no published analogue (APR, lounge counts, caps) are disclosed per card in assumed_fields and counted in the README.
- **300 profiles + 22 hand-written edge cases.** Segments from student to senior with lognormal incomes and score/payment correlations — plus the awkward cases generators never land on: income exactly on a minimum, ±1 rupee, age window edges, gate boundaries, no history, contradictory preferences.
- **Ground truth computed by the engine itself.** Said plainly: this measures spec conformance and pipeline faithfulness, not policy correctness. A 100% here means the pipeline reproduces its own rules through SQL, Python, ranking and the API — which is worth knowing, as long as nobody calls it model accuracy. I label it as conformance everywhere it appears.

7 How I evaluated it

Five independent measurements, because one number would hide which part is weak. n=322 profiles over 120 cards unless stated.

What	Metric	Result
Rules (spec conformance)	Gate agreement	100.0%
	Eligibility agreement (37,352 card checks)	100.0%
	False eligible rate	0.0 — never approved a card the rules reject
Ranking vs. deterministic oracle	Top-1 accuracy	98.2%
	Top-3 hit rate	99.1%
	NDCG@3	98.4%
Explanations (n=25)	Verifier pass rate	100%, 0 retries, 0 fallbacks
Extraction (n=40)	Field accuracy, stated fields	73% (LLM) / 71% (regex)
	Correct blanks — must stay empty	88%

What	Metric	Result
Robustness	7/7 invariants	injection, PII, determinism all hold
Latency, no prose	p50 / p95	161ms / 2.3s

Two results need explaining, not celebrating:

- **Ground-truth agreement is 100% by construction** (same engine wrote the answers). Its value is as a tripwire: any future drift between SQL, rules, and the API shows up here first. I caught a real zip-misalignment bug with it during development.
- **The LLM-only ablation did not run.** All 12 attempts hit free-tier rate limits, and the harness counts failed calls as excluded rather than scoring them correct. Re-running it on a quiet tier is the obvious next experiment — it would put a number on what the rules are worth.

The weakest real stage is spend extraction (~10% on monthly_spend): people say “about 40k” rather than a category breakdown. The system handles it correctly by asking instead of guessing, which is the right behaviour, but it means chat takes more turns than the form.

8 Deployment

every dependency degrades: no key still serves, on SQLite + local fallbacks

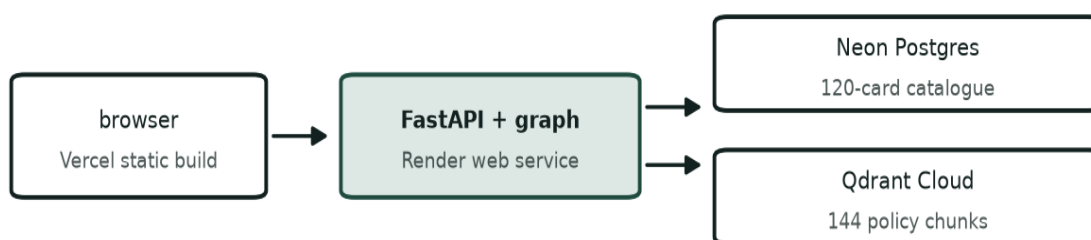


Figure 5 — every dependency degrades, so a partial configuration still serves.

- **Frontend → Vercel:** <https://card-sathi.vercel.app> (Vite static build, VITE_API_URL points at the API).
- **Backend → Render:** <https://card-sathi-api.onrender.com> (render.yaml blueprint, Singapore region to sit near Neon).
- **Catalogue → Neon Postgres. Policy → Qdrant Cloud.**
- Four secrets total: GROQ_API_KEY, OPENROUTER_API_KEY, DATABASE_URL, QDRANT_URL + QDRANT_API_KEY. Everything else has a working local default.

Local run: cp .env.example .env, make setup, make data, make dev. 161 tests (make test), lint (make lint), evaluation (make eval).

9 Limitations and what I would do next

- The catalogue is synthetic and Indian-market-flavoured rather than real. Plugging in a licensed feed would change nothing architecturally — it lands in the same Card schema.
- Latency is dominated by free-tier reasoning models (~25s typical). A smaller extraction model and streaming the explanation would both help; I measured the split before deciding neither was worth it for this build.
- No auth, rate limiting, or persistence on the API. Chat state lives in the client by design, which keeps it stateless but means nothing is stored.
- The verifier checks text against numbers; it does not judge reasoning. The optional LLM grounding check exists, is off by default, and can only add a failure.
- Next: re-run the ablation on a quiet tier, add Hindi/Hinglish extraction tests (the regex handles some; the eval does not measure it), and stream node completions over SSE so the step-by-step UI shows live progress instead of paced narration.

The core claim of this project is small and checkable: no language model anywhere in it can approve a card, change a rank, or invent a number that survives to the customer. Everything else is presentation.

Appendix — mermaid sources

The five figures above were rendered from these sources (matplotlib, checked into scripts/build_report.py). Included so the diagrams stay editable.

trust

```
flowchart TD
    subgraph Trust ["Trust hierarchy – nothing below overrules above"]
        RULES["Business rules  
(plain Python)  
decides everything"]
        DB["Catalogue  
(PostgreSQL)"]
        RAG["Policy prose  
(Qdrant)"]
        LLM["LLM (Groq → OpenRouter)  
prose only"]
        RULES --> DB
        DB --> RAG
        RAG --> LLM
    end
```

pipeline

```
flowchart TD
    A[chat message] --> I[intake: LLM extracts profile]
    I --> F[form profile]
    F --> G[build profile: validate]
    G --> I
    G -->|incomplete| Q[ask one question]
    Q -->|complete| G
    G --> GATE[global gate]
    GATE -->|fail| EVID[evidence]
    EVID -->|pass| PRE[prefilter: SQL narrows catalogue]
    PRE --> EVA[evaluate: Python checks every card]
    EVA -->|none eligible| EVID
    EVA -->|some eligible| RANK[rank: deterministic scoring]
    RANK --> EVID
    EVID -->|retrieve policy evidence| EXP[explain: LLM writes from facts]
    EXP --> VER[verifier]
    VER -->|fail| EXP
    VER -->|pass| OUT[final response]
    VER -->|2 failures| TPL[template fallback]
```

data

```
flowchart LR
    T[tier templates + fixed seed] --> C[120 cards  
Pydantic-validated]
    C --> PG[(PostgreSQL catalogue)]
    PG --> QD[(Qdrant one chunk per card)]
    QD --> P[300 profiles + 22 edge cases]
    P --> GT[ground truth  
computed by the engine]
```